

arb4j — Architecture Review

2026-06-26

Contents

arb4j — Architecture Review	1
1. What arb4j actually is	2
2. The unprecedented capability of the expression compiler	2
3. The architecture, in layers	3
4. The design patterns that organize everything	3
4.1 “Last argument is the result”	3
4.2 Deterministic native ownership; no GC safety net	3
4.3 Intermediates are fields, not locals	4
4.4 The vector is the scalar	4
4.5 Type-directed generation, not runtime polymorphism	4
4.6 Expression strings are literal source	4
4.7 Behaviour on the type, dispatched by polymorphism	4
4.8 Fail loud, never degrade	4
5. How the documents drive the architecture	5
6. How shared functions and variables propagate — the JacobiPolynomialSequence evidence	5
6.1 Shared functions are singletons held in direct typed fields	5
6.2 The parent creates its children and propagates everything downward	6
6.3 Why no cycle ever occurs	6
6.4 Self-recursion: a distinct instance with a shared cache	6
6.5 The only remaining task — mutual recursion the same way, no SCC	7
7. The infinite-sum gap (the live work)	7
8. Open issues — validity triage	8
9. What stinks and what doesn’t — current code vs. best possible	9
9.1 Clean — these are right; do not touch them	9
9.2 Stinks — scaffolding that must be removed or reworked	10
9.3 Open — genuinely unresolved; neither clean nor stink yet	11
9.4 Verdict	11

arb4j — Architecture Review

An assessment of the system’s design, the patterns that organize it, and the capability that has no precedent on the JVM: a compiler that turns written mathematics into verified-arithmetic machine code. This is a conceptual review, not an API catalogue.

1. What arb4j actually is

Two things are fused into one artifact, and the fusion is the point.

The lower half is a **Java binding to FLINT/arb** — arbitrary-precision *ball arithmetic*. A number is not a point; it is a midpoint together with a radius that is a rigorous bound on how far the true value can be from that midpoint. Every operation propagates the radius forward, so the final radius is a *proof* of accuracy: the true answer is guaranteed to lie inside the ball. This is the difference between “the computer printed sixteen digits” and “these digits are certified correct.”

The upper half is an **expression compiler**. It reads mathematical notation as written — Σ , Π , $\int$, ∂ , Γ , ζ , $\sqrt{}$, Greek and blackletter identifiers, sub/superscripts, the indexed-family arrow $i \rightarrow j \rightarrow \dots$ — and emits JVM bytecode that computes it in the ball arithmetic of the lower half.

The fusion matters because neither half is interesting alone. A binding to arb is just another bignum library. A notation-to-bytecode compiler over double would be a toy. Put together, you get a system where you *write the formula the mathematician wrote* and the machine *evaluates it to a provable error bound* at near-native speed. That combination is what is unprecedented.

2. The unprecedented capability of the expression compiler

The claim deserves precision. Plenty of systems evaluate symbolic expressions — Mathematica, SymPy, every CAS. What none of them do, and what arb4j does, is the conjunction of all of the following in a single pass:

1. **The source language is the mathematics itself.** Not an AST built by API calls, not a string DSL that approximates notation — the literal Unicode a working analyst writes. $\Sigma k \rightarrow x^k \{k=0.. \infty\}$ is the program text. The parser treats Σ as an operator, $\rightarrow$ as family abstraction, ∞ as a numeric literal. There is no translation layer where notation degrades into `sum(lambda k: ...)`.
2. **The target is verified arithmetic, not floating point.** The emitted bytecode computes balls. A compiled expression doesn't return a number that is *probably* right; it returns an enclosure that is *provably* right to the requested precision, because every arb operation it calls is a verified operation.
3. **The output is a real JVM class, not an interpreter walk.** Each defined function becomes a genuine bytecode class implementing a typed Java interface. Evaluating it is ordinary method dispatch into compiled code — the AST is gone by runtime. There is no tree-walking evaluator in the hot path. This is what makes it fast enough to be the *implementation* of a research numerical method, not merely a convenience for entering one.
4. **Definitions compose and recurse like a programming language.** A function can reference other functions, itself, or an indexed family of itself, and those references compile to field accesses and method calls between the generated classes. A three-term recurrence for Chebyshev polynomials, a self-referential Müntz coefficient sequence, a mutually-defined cluster — these are written as definitions and become cooperating compiled classes.
5. **Precision is a runtime parameter, not a compile-time constant.** The same compiled class evaluates at 64 bits or 4000 bits because precision (bits) threads through every generated call as an argument. You compile once and choose accuracy per call.

No other system on the JVM — and very few anywhere — closes all five at once. That is the capability the rest of the architecture exists to serve.

3. The architecture, in layers

The system is honestly layered, and each layer has exactly one job.

Native layer (SWIG/FLINT). The C structs become Java handles. A `Real` is a midpoint+radius struct; a `Complex` is two of those; a `Magnitude` is the radius type itself (an unsigned floating bound). The Java object is a thin handle holding a raw pointer (`swigCPtr`) and an ownership flag. The hand-written mathematics — the actual API a user touches — lives in `%typemap(javacode)` blocks in the `.i` files and is *generated into* the `.java` files. The generated `.java` is therefore not the source of truth; the `.i` file is. This is the single most important operational fact about the lower half: edit the `.java` and your change evaporates at the next build.

Type layer. A small, deliberate hierarchy: ball types (`Real`, `Complex`) that carry a radius, and exact types (`Integer`, `Fraction`, `AlgebraicNumber`) that do not. Polynomials and matrices over both. The exact/ball distinction is not a detail — it is a fault line that runs all the way up into the compiler and determines, for instance, what “has this sum converged?” can even mean.

Compiler layer. Parser → AST of typed nodes → bytecode via ASM → a classloader that defines the class → an instance with its references wired in. The codomain type drives the bytecode: the same Σ node emits `arb_add` calls for a real sum and `acb_add` calls for a complex one, chosen at *generation* time so the *generated* code has no type dispatch in it at all.

Function/math layer. Typed interfaces (`RealFunction`, `ComplexFunction`, `Sequence`) that generated classes implement, plus the hand-written mathematics — orthogonal polynomials, zeta/xi/Hardy-Z, the fractional-Riccati Müntz-Padé machinery — that consume them.

The layering is clean in the sense that each layer only knows about the one below it, and the compiler’s product is indistinguishable from a hand-written member of the function layer. A compiled Chebyshev sequence and a hand-coded one implement the same interface and are used the same way. That substitutability is a sign the abstraction boundaries are real.

4. The design patterns that organize everything

These are the recurring decisions. Each one appears in dozens of places and together they *are* the house style.

4.1 “Last argument is the result”

Every operation writes into a caller-supplied destination and returns it: `x.add(y, bits, out)`. This mirrors `arb`’s C convention exactly, and it exists because native memory allocation is the dominant cost. By letting the caller own and reuse the output buffer, the hot path allocates nothing. The two-argument form `x.add(y, bits)` writes into this. The bytecode emitter *targets this shape* — it knows the trailing argument is the destination — so the convention is not stylistic, it is load-bearing for code generation.

4.2 Deterministic native ownership; no GC safety net

The SWIG finalizers are deliberately deleted. Nothing is freed at GC time. Every ball type is `AutoCloseable` and *must* be closed, or its native allocation leaks until the process exits. This sounds dangerous and is in fact the correct choice: finalizer-based cleanup is non-deterministic and would interleave native frees unpredictably with computation. The discipline instead is structural — generated classes emit a `close()` that disposes every intermediate field, and the

“intermediates are fields, not locals” decision (below) exists largely so that this generated `close()` can find and release them.

4.3 Intermediates are fields, not locals

When the compiler needs scratch space — a loop index, an accumulator, an operand value buffer — it allocates a *field* on the generated class, not a JVM local. The reason is ownership: a field can be closed by the generated `close()`; a local cannot. This single decision explains a great deal of the compiler’s shape, including why the Σ/Π loop index is a field and why every operation node names and registers its temporaries.

4.4 The vector *is* the scalar

A `Real` is not a scalar that you put into an array to get a vector. A `Real` holds a contiguous native buffer of `dim` ball elements, with `dim == 1` for a scalar. Indexing is `get(i)`; the vector is one handle, one allocation, one owner. Java arrays of `Real` are forbidden precisely because they would shatter the single-owner model, double the allocation, and defeat the contiguous-buffer layout the code depends on. This is unusual and worth internalizing: the type is already the vector.

4.5 Type-directed generation, not runtime polymorphism

The compiler decides at generation time what concrete type a node produces (`generatedType`) and emits type-specific calls. A real codomain compiles to real operations; a complex one to complex operations. The *generated* bytecode contains no `instanceof`, no type switch — the branching happened once, in the compiler, and was burned into the instructions. This is why “dispatch on the codomain type in the generator” is not a violation of the no-special-casing principle: it is the opposite of runtime special-casing.

4.6 Expression strings are literal source

An expression body is source code. It is parsed into an AST and compiled. It is therefore never built by concatenation, formatting, or substitution at runtime — doing so would be like generating Java by string-pasting identifiers. When a naming collision tempts you toward string manipulation, the architecture’s answer is a per-instance context or a shared compiled instance through a field, never textual surgery on the formula.

4.7 Behaviour on the type, dispatched by polymorphism

Operations that transform the AST construct the *same concrete subclass* they were called on, so the type — and therefore the behaviour — survives the transformation. There are no `instanceof` ladders; the JVM’s own dispatch is the mechanism. This keeps node behaviour local to each node class.

4.8 Fail loud, never degrade

Unimplemented paths throw; they do not return null and they do not silently fall back to a different algorithm. The “no fallback” rule is strict to the point of banning the *word* in design notes, and for a verified-arithmetic library it is the only coherent stance: a fallback is an undocumented change of algorithm that hands the user a number from a different method than they think they’re using, with no guarantee the two agree. Either the method works on its claimed domain and proves it, or it refuses the input.

5. How the documents drive the architecture

The docs/ papers are not background reading; they are the *requirements document*. They describe one interconnected body of mathematics — the constant/polynomial-coefficient **fractional Riccati equation**, its appearance as the **rough-Heston** characteristic function, and the **Müntz-Padé spectral tau** method, **Riemann-Hilbert** continuation, and **Toda lattice** that are presented as equivalent faces of the same object. The library exists to be the high-performance, *verified* implementation of this mathematics.

That mathematics is saturated with infinite series. The Riccati solution is a Puiseux series $y = \sum a_k t^{\{k\mu\}}$ whose coefficients obey a Gamma-ratio recurrence with an inner Cauchy convolution. The cumulant generating function is $\sum_k d_k(v) T^{\{k\mu+1\}}$. The orthogonal-polynomial and continued-fraction machinery exists to turn those infinite series into *finite, error-bounded* objects.

This is the through-line that explains the compiler’s design pressure:

- **Indexed families** ($a: k \mapsto v \mapsto \dots$) exist because the coefficients *are* an indexed family of functions of the parameter, defined by recurrence.
- **Self-reference** exists because a_k is defined in terms of a_j for $j < k$.
- **Finite $\Sigma\{j=1..k-2\}$** exists because the Cauchy convolution is structurally bounded and must stay bounded.
- **The push toward $\Sigma\{k=0..\infty\}$** exists because the *outer* series are genuinely infinite and currently faked by hand-written Java loops with ad-hoc stopping rules — exactly the thing the compiler should own.

The quality bar set by the papers is “rigorous and verified,” and the library is built to meet it: ball arithmetic carries provable enclosures, bytecode is verified at generation, results are “certified to requested precision.” The papers prove convergence *analytically*; the library is meant to deliver the *computational* counterpart with the error bound attached.

6. How shared functions and variables propagate — the JacobiPolynomialSequence evidence

The recursion machinery is **not accidental** — it is a deliberate, elegant mechanism, and the acyclic case proves it. The single task ahead is to make mutual recursion work the *same* way, without the SCC detection layer. The evidence is in the decompiled classes generated from JacobiPolynomialSequence, whose six definitions share one Context(α, β):

```
C: n → 2n + α + β          F: n → C(n-1)C(n)
A: n → (F(n)x + α² - β²)(C(n) - 1)/2    E: n → n · C(n/2) · C(n-1)    B: n → (n+α-1)(n+β-1)C(n)
P: n → when(n=0, 1, n=1, (C(1)x - β + α)/2, else, (A(n)P(n-1) - B(n)P(n-2))/E(n))
```

The reference graph is the DAG $C \rightarrow F \rightarrow \{A, E, B\} \rightarrow P$, plus P’s self-edge $P(n-1), P(n-2)$. The decompiled P.class and A.class show *exactly* how the shared instances are propagated, and it is concise, optimal, and cycle-free.

6.1 Shared functions are singletons held in direct typed fields

A does not hold a generic map of dependencies — it holds **direct, concrete typed fields**: public F F; public C C;. P holds public P P; public E E; public B B; public A A; public C C;. Because the field types are the concrete generated classes, every call in evaluate_body is an ordinary virtual dispatch — this.A.evaluate(...), this.C.evaluate(...) — with **no name lookup in the hot path**. The AST is gone; what remains is method calls between objects.

6.2 The parent creates its children and propagates everything downward

The wiring model is **strictly top-down**. The context injects references into **one** instance — the externally-constructed top-level element — and then calls `initialize()` on it. From there, nothing calls back into the context:

- `initialize()` **creates** each child instance it directly references (`this.C = new C();`), and propagates the live context down into it (`this.C.context = this.context;`).
- It propagates the **shared variables** down by reference (`this.A. $\alpha$  = this. $\alpha$ ;`), never copying.
- It propagates the **shared sub-functions** it owns down into the siblings that need them: `this.A.C = this.C; this.B.C = this.C; this.E.C = this.C;`. One pointer assignment per shared edge.

Each child, on its own first `evaluate()`, runs *its* `initialize()` and does the same for *its* children. The single shared C arises because the **parent creates one C and pushes it into A, B, E** — and because every `create-child` step is guarded by `if (this.<field> == null)`, a value the parent already propagated is never overwritten. So when `A.initialize()` later runs, `A.C` is already non-null and A does not make its own.

This is the whole mechanism: **construct downward, propagate by reference, null-guard so parent wins**. A function instance never needs to consult a registry, because the only instance that ever has to be told its references from outside is the top-level one, and the context tells it exactly once.

The `lookupFunctionInstance` callback is a kludge that violates this and must be removed. The currently-generated `initialize()` contains a middle branch — `Function f = this.context.lookupFunctionInstance("C"); if (f != null) this.C = (C) f;` — added later (issue #1032, commit `ed791c72`) for the σ /m/a Müntz cluster. It makes a child phone home to the context for an instance the parent should have propagated. It is redundant in the DAG case (the parent's `this.A.C = this.C` already wired it, so the lookup branch is dead) and it is the wrong answer in the mutual case (it papers over a propagation that wasn't emitted). The clean emitter — *create-if-null + propagate-context-down, nothing else* — existed before this and is what the generated code should return to.

6.3 Why no cycle ever occurs

`initialize()` **only allocates and wires fields — it never evaluates**. Building the object graph is pure pointer-plumbing: create each dependency, set its shared variables, set its shared sub-functions, mark `isInitialized`. Nothing in `initialize()` calls `evaluate`, so constructing the graph *terminates regardless of reference topology*. Evaluation happens later, lazily, and is memoized. The `isInitialized` guard makes it idempotent.

6.4 Self-recursion: a distinct instance with a shared cache

P's self-reference is the subtle, beautiful part. P does **not** set `this.P = this`. It does:

```
this.P = new P();
this.P.context = this.context;
this.P.cache   = this.cache;    // SHARED memoization
```

A *distinct* instance, but pointing at the *same* `IndexCache`. The consequence is both correctness properties at once:

- **Re-entrance safety:** the recursive call `this.P.evaluate(n-1)` runs in a different object, so it uses *that object's* scratch temporaries (`vXR0011`, ...) — it cannot clobber the in-

progress outer call’s temporaries. This is why the synchronized + evaluating guard is a tripwire that *never fires* here: the design avoids re-entering the same instance.

- **Optimal memoization:** because the cache is shared, each index $P(k)$ is computed once and reused across the entire recursion (`Function.peek/poke`).

The `Context.injectFunctionReferences` comment states the rule directly: *never alias* `f.<name> = f`; a function’s own-type field is the recursive-call receiver and must be a different instance, or recursion re-enters `f` and smashes its scratch.

6.5 The only remaining task — mutual recursion the same way, no SCC

Everything above is *general*. Look at what mutual recursion ($a \leftrightarrow S$, each referencing the other) actually needs versus what the DAG already does:

DAG (A ’s <code>this.F.C = this.C</code>)	Mutual ($a \leftrightarrow S$)
allocate each instance once via context	allocate a and S once via context
wire shared sub-functions by field assignment	$a.S = S$; $S.a = a$;
distinct self-instance + shared cache	distinct peer-instances + shared caches

The mutually-recursive case adds **nothing** but a cycle in the *reference graph* — and the reference graph’s shape is irrelevant to wiring, because wiring is just pointer assignment performed *after* all instances exist. **Allocate-all-then- wire-all** handles a DAG and a cycle *identically*; you never have to know which one you have.

That is precisely why the SCC/Tarjan detection, the `selfReferential` sidecar, the `isInCycle` predicate, and the reflective post-hoc `backfillMutuallyRecursiveReferences` pass are unnecessary. They exist to *detect* a cycle and *patch references after the fact* — solving a problem that vanishes the moment you separate allocation from wiring, exactly as `JacobiPolynomialSequence` already does for its DAG and its P self-loop. The genuine, elegant mechanism is already in the generated code; the task is to let the mutual case use it unchanged instead of routing it through cycle-detection scaffolding.

7. The infinite-sum gap (the live work)

The system already parses ∞ (it is a numeric literal, resolving to the runtime infinity constant) and already represents an unbounded upper limit in the AST. What it lacks is a compiler that *does the right thing* with it. The bounded $\Sigma\{k=a..b\}$ compiles to a clean counted loop. The unbounded case has only a half-built stand-in: $a \sim$ suffix that hands the whole loop to a Java-level accumulator object using a wrong tolerance (half the requested precision) and a forbidden “keep the best partial” fallback, and only for real sums.

The correct design follows directly from the architecture’s own principles:

- **It belongs inline in the loop**, not in a side object — same counted-loop skeleton, with a convergence exit added.
- **The criterion is pure ball arithmetic**, available only to ball types: stop when the magnitude of the just-added term is within the *radius* of the partial sum. At that point the term is below the partial sum’s own certified uncertainty, so adding more cannot improve the enclosure. For complex sums the norm is compared to the Euclidean ball radius. No bits appears in the criterion; bits governs term evaluation only.
- **It is type-directed, not type-special-cased:** the generator emits the real-radius comparison for a real codomain and the complex-radius comparison for a complex one, with no runtime branch in the result.

- **Exact-type infinite sums are a genuinely different question** — there is no radius, so the ball criterion cannot apply — and the honest position is that the exact case is unresolved and must not be force-fit to either a fake tolerance or a premature exception.

This is the one place where the architecture is visibly mid-stride, and the open issues (the #103 decomposition and its arb4j successors) are the roadmap for finishing it.

8. Open issues — validity triage

Seventeen issues are open. Read against the architecture, they sort cleanly, and the axis the request asks for — valid / invalid / already-fixed — resolves as: **none are already fixed**. Every open issue describes genuinely unfinished work; two have *partial in-tree workarounds* (noted below) but no issue is a resolved-yet-unclosed stragler. The grouping:

Valid and central to the infinite-sum effort — #1053, #1054, #1045. Σ must own its own accumulation in one evaluation pass (#1045); the unbounded case must terminate by a *proven* convergence criterion or throw on detected divergence and **never iterate forever** (#1053); and continued-fraction / Padé-diagonal resummation is the sophisticated answer for slowly- or conditionally-convergent series (#1054). The mechanism spread across them — a term-vs-radius tolerance loop versus a Wynn- ϵ / Padé resummation — is a real design choice, not a contradiction. These are the project correctly identifying its own unfinished work (§7).

Valid and corrective — the recursion teardown — #1055, #1056, #1048, #1047. Remove backfillMutuallyRecursiveReferences in favour of deterministic allocate-all-then-wire-all (#1055); enable the evaluate() re-entrance guard by default (#1056); replace SCC instance-aliasing with a fresh instance per peer plus a value-layer (name, index, bits) cache (#1048); fix the 10 generator sites that re-enter evaluate() on the same instance (#1047). These are exactly the accidental-complexity removal of §6 and §9.2. The guard surfacing ten failing sites is not ten bugs; it is one design defect — recursion descending into the *same* instance — made visible in ten places.

Valid bugs, independent of the sum work — #1038, #1050. The AST differentiator treats parameter-bound coefficients (p, q, r) as constants and drops $\partial \text{coeff} / \partial \text{param}$ (#1038); cache invalidation leaves stale memoized balls after a parameter perturbation (#1050). Both are real; both are *partially* mitigated by in-tree workarounds (a hand-built linearized recurrence; manual cache clearing) rather than fixed — this is the closest anything comes to “already addressed,” and it is only partial.

Valid performance work — #1041. The Riccati Müntz-Padé hot path is compilation/allocation-bound, not arithmetic-bound; the issue ranks concrete optimizations. Legitimate, measured, and off the correctness critical path.

Valid enhancements, not on the critical path — #1022, #1024, #1026, #1028. Robust Context merge (#1024), a matrix DSL for the expression language (#1026), multivariate-polynomial (acb_mpoly) wrapping (#1022), and a HasComplexity cost-generating-function interface (#1028). Design-stage, legitimate, independent.

Near-duplicates — #1043 $\subset$ #1044. Both concern removing a hard cap on cumulant/epsilon order so Wynn- ϵ terminates only on convergence; #1043 (EpsilonTable cap) is the narrower subset of #1044 (the CUMULANT_ORDER=48 cap plus E1/E3 violations in the pricer).

Invalid / inert — #1037. Title “misc.”, body an ellipsis. No actionable content.

A scoping caveat worth stating plainly: the original #103 sub-issue decomposition targets the *downstream* project (bonanzai), not arb4j; the arb4j issues above are its restatement and evolution. Several “valid” items are partly bonanzai’s responsibility, and the rough-Heston

pricing-table integration sits across both. Triaging them as if all were arb4j-local would overstate arb4j's own backlog.

9. What stinks and what doesn't — current code vs. best possible

Everything above describes what the code *does*. This section judges it. For each subsystem: what it does now, what the best possible design is, and a verdict. The test for “stinks” is the project's own: does the construct exist because the mathematics demands it, or because an author didn't understand the machinery and bolted on scaffolding to compensate? Scaffolding that re-implements, badly, what the design already gives for free is the thing that stinks.

9.1 Clean — these are right; do not touch them

Subsystem	What it does now	Why it is already the best possible
Native ball core (Real, Complex, Integer, Float, Magnitude, polynomials, matrices)	One Java handle per FLINT/arb struct; the type is the vector (<code>dim ≥ 1, get(i)</code>); <code>AutoCloseable</code> owns the native memory	There is no leaner faithful mapping of a C struct that already holds a contiguous element array. A <code>Real[]</code> would double-allocate and break single-handle ownership. The model matches the library it wraps.
Last-arg-is-result (<code>x.sqrt(bits, out)</code>)	Trailing destination is written and returned; omit it for a fresh receiver	One convention covers in-place and functional styles, lets the emitter pick the destination to avoid clobbering, and needs no allocation discipline beyond the closeable itself.
Intermediates are owned fields, not locals	Every sub-result is a field, disposed in the generated <code>close()</code>	Makes deterministic disposal mechanical and total — the class closes exactly what it owns. Locals would leak native memory or need a shadow free-list.
Type-directed generation	The generator emits Real-typed or Complex-typed bytecode from <code>generatedType</code> ; no runtime type switch survives into the product	The decision is made once, at compile time, where the type is known. The product is monomorphic and as fast as hand-written code. This is the source of the compiler's whole advantage.
Expression strings are literal source	<code>express("x→...")</code> parses to AST→bytecode; the string is never assembled at runtime	Keeps the notation authoritative and auditable; forbids the class of bugs where a renamed identifier silently changes the mathematics.

Subsystem	What it does now	Why it is already the best possible
Fail loud, never degrade	Out-of-domain inputs throw; no method silently swaps algorithms	The only honest stance for verified arithmetic: a returned ball must enclose the truth or there must be no returned ball.
Recursion construction (initialize() = create-children + propagate-down; distinct self-instance + shared cache; lazy memoized evaluate)	Parent allocates each child, shares variables and sub-functions by reference, guards with if field == null; self-reference is a separate instance sharing the memo	This is the elegant core the project is built around. Graph construction never evaluates, so it terminates for any topology; sharing is one pointer assignment per edge; self-recursion is re-entrance-safe because the inner instance has its own temporaries. Nothing about it is accidental.

9.2 Stinks — scaffolding that must be removed or reworked

Context.lookupFunctionInstance (the pull-from-context callback). *Now:* each generated initialize() can phone home — `f = context.lookupFunctionInstance("C"); if (f != null) this.C = (C) f;` — to fetch an instance. *Best:* the parent already created the one `C` and propagated it (`this.A.C = this.C`), so the child never needs to ask anyone; the only instance the Context injects into is the single top-level element, exactly once. *Why it stinks:* it is **dead code in the DAG case** (the field is already non-null from parent propagation) and **the wrong answer in the cyclic case** (it masks a propagation that should have been emitted). It inverts the design’s one firm rule — construction flows downward — into a runtime upward dependency on a registry.

The SCC subsystem (`selfReferential`, `populateFunctionReferenceGraph`, `stronglyConnectedComponents/T`, `sccByName`, `sccOf`, `isInCycle`, `warmToBottomUp`, `sccWarmOrder`, `warmMember`, and `Expression.backfillMutually`). *Now:* ~250 lines that build a function-reference graph, run Tarjan to find cycles, then *manually warm* each cycle’s members bottom-up in dependency order, driven by externally-registered per-member “k-extent providers,” with a `CompilerException` if one wasn’t registered. *Best:* allocate every instance in the cluster, wire the peer pointers (`a.S = S; S.a = a`), share caches, and let the **existing lazy, memoized evaluate** compute each value once on demand. Mutual recursion needs nothing the DAG case doesn’t already have — two pointer assignments instead of one. *Why it stinks:* it re-implements, laboriously and with a new failure mode, what construct-downward + memoized evaluation already does for free. It treats a cycle as a categorically special structure requiring graph theory and a hand-rolled evaluation schedule, when “allocate-all-then-wire-all” makes a cycle and a DAG **identical** to the constructor. It is the canonical symptom of not trusting the recursion machinery — and it even leaks the project’s own banned vocabulary (“curried”) into comments and a runtime error string, which is the tell that the misconception is baked in.

ConvergentSeriesAccumulator and the ~ sigil. *Now:* $\Sigma\{k=a..b\}$ hands the loop to a separate Java object that runs its own loop with tolerance $2^{-(\text{bits}/2)}$, keeps a “best” partial as a fallback, and only handles real sums. *Best:* ∞ is just a different termination test on the *same* counted loop; the convergence exit is inline — stop when $|\text{term}| \leq \text{rad}(\text{partialSum})$ — and it is type-directed for Real and Complex with no side object. *Why it stinks:* it is a **parallel evaluator** (a second place the sum is computed, which can disagree with the primary one), its tolerance is a **fabricated number** unrelated to the certified ball radius, and its “keep the

best” branch is a **forbidden fallback** — transparent degradation in a library whose entire reason to exist is non-degraded verified answers.

9.3 Open — genuinely unresolved; neither clean nor stink yet

- **Exact-type (Fraction/Integer) infinite-sum convergence.** There is no radius, so the ball criterion cannot apply, and convergent rational series (ζ at integers) demonstrably exist. The honest state is *unresolved*: it must not be force-fit to `isZero`, a fake tolerance, or a premature exception. The ball-type path can ship independently while this is worked out.
- **Where mutual-recursion wiring is emitted once the SCC layer is gone.** Removing the kludge means the parent’s `initialize()` must emit the peer pointer assignments for cyclic clusters the same way it already emits `this.A.C = this.C` for the DAG — this is a concrete, bounded task, not a research question, but it is not yet done.

9.4 Verdict

The core is genuinely without precedent on the JVM and is built right: a verified-arithmetic native core with deterministic ownership, and a compiler whose product is real bytecode indistinguishable from hand-written code, organized by a small set of mutually reinforcing patterns all serving one goal — *write the mathematics, get a proof-carrying answer at machine speed*. The recursion construction at its center is elegant and deliberate.

Everything that stinks is **localized, recent, and additive** — a pull-from-context callback, an SCC-and-warming layer, and a convergent-sum side object — each one a case of an author not trusting machinery that was already sufficient and adding scaffolding to compensate. None of it is load-bearing for the good design; all of it can be deleted in favor of the simpler thing the architecture already implies. The work ahead is **subtraction, not invention**: remove the scaffolding, finish the inline infinite sum, and let the original construction model carry mutual recursion exactly as it already carries the DAG.